

```

        ad = get_object_or_404(SeekingAd, id=ad_id,
                               owner=request.user)
        form = SeekingAdForm(instance=ad)
    else: # POST
        if ad_id == 0:
            form = SeekingAdForm(request.POST)
        else:
            ad = get_object_or_404(SeekingAd, id=ad_id,
                                   owner=request.user)
            form = SeekingAdForm(request.POST,
                                  instance=ad)
    ...

```

The same query as in the GET case of Edit mode →

The instance argument prepopulates the form based on the corresponding model. ←

In Edit mode, the view looks up the existing SeekingAd object. Note that the query requires both the ID and the owner. ←

Ad ID of 0 means the form is created based solely on the submitted data. ←

Populate the form with both the submitted data and the instance information from the corresponding model. ←

WARNING The following is important security content!

As ads belong to users, only those users should be able to edit the ad. Merely taking the ID of an ad, looking it up, and allowing someone to edit the content is problematic. The query in the `get_object_or_404()` call takes two arguments: both the ID of the `SeekingAd` and the logged-in user. If the logged-in user manually changes the URL to attempt to edit an ad they don't own, a 404 exception gets raised.

A better version of this code would allow an admin user to edit any ad, which can be done with an additional check to `request.user.is_staff`. I leave this as an exercise for the reader. (I've always wanted to write that sentence, and it is quite literally true. This is one of the chapter's exercises.) The route registration for the two variations on your view is no different from any other registration you've done, except this time, two different routes point to the same view function:

```

# RiffMates/content/urls.py
...
urlpatterns = [
    ...
    path("seeking_ad/", views.seeking_ad,
          name="seeking_ad"),
    path("edit_seeking_ad/<int:ad_id>/", views.seeking_ad,
          name="edit_seeking_ad"),
]

```

The seeking_ad view in Add mode ←

The seeking_ad view in Edit mode with its required ad_id argument ←

The `list_ads.html` file needs a small tweak: it needs Edit links for ads owned by the logged-in user. When you call `render()` from a view, the `request` object you pass in gets included in the template's context dictionary. That means anything in the request can be accessed within your template. The following code goes inside the `{% for %}` loop where an ad gets rendered, displaying a link if the ad's owner is the `request.user`:

```

<!-- RiffMates/templates/list_ads.html -->
<!-- ... -->
{% if ad.owner == request.user %}
    <a href="{% url 'edit_seeking_ad' ad.id %}">Edit</a>
{% endif %}

```